

SERVICE ACTUEL · APP-MANAGEMENT

Applications, images et espaces de développement

10 capacités analysées

services/app-management · Python, FastAPI, SQLAlchemy/SQLAlchemy, PostgreSQL/Alembic, FastStream/Kafka, Kubernetes, Helm, Kaniko et Trivy.

Provisionne les applications Kubernetes, leurs images, instances, dépôts et devspaces ; fournit aussi à process-data un adaptateur de sessions Python/PySpark.

Lecture de code

Pas de qualification runtime nouvelle

Domaines cible : Applications, Runtimes, IAM, Egress, Audit, Migration, Maintenance.

Périmètre de lecture

- Routers applications, instances, lifecycle, images, sessions privées, dépôts internes/externes et devspaces.
- Services create_application, deploy_instance, cancel_deployment, create_devspace, image_build et internal_repository_file.
- Adaptateur SessionRuntimeServiceImpl et instructions de chiffrement, secrets SSH, Kafka et migrations.

Objets et cycle de vie

- Application/Namespace : identité, projet, propriétaire et configuration désirée.
- Instance et DeploymentExecution : configuration et historique de déploiement, principale, annulation et dates.
- Registry/Image/Build : source Git, commit, image, credentials chiffrés et scans.
- Repository : dépôt interne/externe, projet, visibilité et liens applicatifs.
- Devspace/SSHKey : espace personnel, instance et clés.
- Outbox d'autorisation, miroirs utilisateurs et tombstones projets.

Dépendances du service

- IAM/APISIX pour identité, droits et faits de relation.
- PostgreSQL pour entités, chiffrement et historique ; Kafka/FastStream pour demandes et résultats.
- Kubernetes/Helm pour les workloads, Kaniko/Trivy et registre OCI pour les images.
- Gitea ou autres fournisseurs Git ; process-data pour sessions de calcul.

Fonctionnalités et compatibilité à préserver

AM-01 Cible non qualifiée

Catalogue d'applications, propriété et rattachement au projet

COMPORTEMENT OBSERVÉ

Les routes créent, listent, modifient et suppriment les applications, vérifient leur existence et transfèrent leur propriétaire. La création lie une application à son namespace et éventuellement à un projet. Elle vérifie la cohérence du projet dans les arguments, refuse un projet supprimé et sérialise les opérations avec des verrous par propriétaire, projet et application. Les liens vers des applications agents sont autorisés avant création ; une cible absente ou refusée échoue sans laisser un lien. Les faits d'autorisation sont déposés dans l'outbox pour les applications concernées.

UTILISATEURS ET RÔLES

Développeur, responsable de projet et administrateur.

INTERFACES DANS LE CODE

Application CRUD router : /apps ; CreateApplicationService.create_application ; update_application_owner.

DONNÉES ET ÉTATS

ApplicationEntity, NamespaceEntity, lien application-agent, propriétaire, project_id et tombstone de projet.

DÉPENDANCES ET ÉCHANGES

IAM/gateway, outbox SQL et Kubernetes par les déploiements ultérieurs.

COMPORTEMENT CIBLE PROPOSÉ

Applications Rust possède application, rattachement, propriétaire et configuration désirée. IAM reste l'autorité d'accès ; la création et ses faits utilisent une transaction et une clé idempotente.

MIGRATION DE CETTE CAPACITÉ

Importer IDs et rattachements ; préserver les verrous contre création concurrente/offboarding et la validation de chaque cible. Garder une façade HTTP compatible pendant la bascule.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Créer une application pendant la suppression de son projet : elle est refusée ou appartient explicitement au résultat ordonné de l'opération, sans namespace orphelin ni droits tardifs.

Références de l'analyse

[monorepo · services/app-management/src/app_manager/application/router/application_crud_router.py:158](#) Contrat de création et autres commandes du router.

[monorepo · services/app-management/src/app_manager/domain/service/application/create_application_service.py:51](#) Validation, verrous, namespace et outbox.

AM-02 Cible non qualifiée

Instances, configuration déployable et instance principale

COMPORTEMENT OBSERVÉ

Une application possède plusieurs instances dont le détail, la liste, la configuration et les déploiements sont consultables. Le router fournit création, mise à jour, suppression et changement de statut principal. Les dépendances de route contrôlent la parenté application/instance ou application/image afin qu'un identifiant valide d'une autre application ne soit pas accepté sous un parent arbitraire. Les ressources et historiques de sévérité ont des lectures dédiées.

UTILISATEURS ET RÔLES

Développeur applicatif et opérateur.

INTERFACES DANS LE CODE

Instance router : create_instance, update_instance, update_instance_main_status, list_instance_deployments ; enforce_path_parentage.

DONNÉES ET ÉTATS

InstanceEntity, application parente, image, configuration, historique de deployment et sélection principale.

DÉPENDANCES ET ÉCHANGES

Applications, images, autorisation de gateway et ressources Kubernetes.

COMPORTEMENT CIBLE PROPOSÉ

Applications Rust gère une révision immuable de configuration par déploiement, une référence de digest d'image et une sélection principale explicite. Le namespace n'est pas une preuve de droit.

MIGRATION DE CETTE CAPACITÉ

Importer les instances et leur parenté, traduire les secrets en références et conserver les IDs d'historique. Qualifier les contraintes de principale et la concurrence avant activation.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Changer deux fois simultanément l'instance principale conserve un résultat cohérent. Une instance d'une autre application ne peut être lue ou déployée par substitution de parent.

Références de l'analyse

[monorepo · services/app-management/src/app_manager/application/router/instance_router.py:84](#) Création et cycle de configuration.

[monorepo · services/app-management/src/app_manager/application/router/instance_router.py:221](#) Sélection de l'instance principale.

[monorepo · services/app-management/src/app_manager/application/router/parent_scope.py:100](#) Validation de la parenté du chemin.

AM-03 **Cible non qualifiée****Déployer, arrêter, reprendre et annuler une instance****COMPORTEMENT OBSERVÉ**

Le lancement valide l'instance et son état puis contrôle que l'utilisateur peut employer le compte de service configuré. Il crée une exécution de déploiement, passe l'instance à DEPLOYING, synchronise le statut application et commit avant d'émettre la demande. Une précédente demande encore pending peut être marquée CANCELED avant la nouvelle. L'annulation charge le déploiement et exige son état annulable ; elle conserve la date et le résultat en base. Les routes offrent également start, stop et logs. L'émission après commit doit être distinguée d'une transaction unique avec le transport.

UTILISATEURS ET RÔLES

Propriétaire autorisé et exploitation.

INTERFACES DANS LE CODE

InstanceLifecycleRouter.deploy_instance, stop_instance, start_instance, cancel_deployment, get_instance_logs.

DONNÉES ET ÉTATS

DeploymentExecution incrémental, états d'instance/application et dates d'annulation.

DÉPENDANCES ET ÉCHANGES

Consumer Kafka, Helm/Kubernetes, compte de service IAM, watchers de déploiement.

COMPORTEMENT CIBLE PROPOSÉ

Applications/Runtimes Rust utilisent des commandes persistées, reçoivent des effets Helm/Kubernetes et réconciliation de l'état désiré. Une annulation a une phase précise ; un timeout ne vaut pas preuve d'arrêt.

MIGRATION DE CETTE CAPACITÉ

Introduire un reçu durable couvrant commit puis publication d'événement. Reprendre les déploiements en attente ou les drainer avant de changer le propriétaire de l'orchestration.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Couper le service après commit et avant émission : le déploiement reste reprenable et s'exécute une seule fois logiquement. Annuler une demande déjà exécutée retourne son état réel.

Références de l'analyse

[monorepo · services/app-management/src/app_manager/domain/service/instance/deploy_instance_service.py:58](#) Validation, compte de service et lancement.

[monorepo · services/app-management/src/app_manager/domain/service/instance/deploy_instance_service.py:102](#) Commit avant événement et états.

[monorepo · services/app-management/src/app_manager/domain/service/instance/cancel_deployment_service.py:35](#) Annulabilité et persistance.

[monorepo · services/app-management/src/app_manager/application/router/instance_lifecycle_router.py:65](#) Lecture des logs et lifecycle public.

AM-04 Cible non qualifiée**Construire, reconstruire et inspecter une image****COMPORTEMENT OBSERVÉ**

La construction à partir d'un dépôt vérifie l'application et son lien vers ce dépôt, résout la référence Git en commit et crée un build. Un dépôt non lié répond comme absent. Le service réutilise les entités registre/image/credentials, commit le build puis publie ses événements. Le rebuild retrouve le build précédent et sa source. Le router expose détail, historique, vulnérabilités et logs ; pour un build actif, les logs sont streamés depuis le pod, sinon lus dans les logs conservés. Le service ferme la transaction de lecture avant le flux long.

UTILISATEURS ET RÔLES

Auteur d'application et administrateur des environnements.

INTERFACES DANS LE CODE

Image router : create_image, rebuild_image, list_image_builds, get_image_vulnerabilities, get_build_logs ; ImageBuildService.

DONNÉES ET ÉTATS

BuildEntity, commit source, image/registre, historique et résultats de scan.

DÉPENDANCES ET ÉCHANGES

Git, Kaniko, Trivy, registre OCI, Kubernetes et événements Kafka.

COMPORTEMENT CIBLE PROPOSÉ

Applications/Runtimes Rust gèrent la spécification du build et son reçu ; les exécutants sans JVM restent encapsulés. Le déploiement référence un digest qualifié avec provenance, SBOM et audit.

MIGRATION DE CETTE CAPACITÉ

Conserver commits, builds et diagnostics ; épingler le digest avant déploiement et qualifier l'idempotence de la demande. Reprendre les packages Python personnalisés via le registre d'environnements Runtimes.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un rebuild produit un nouveau reçu sans écraser l'historique. Une réponse perdue ne crée pas deux effets non corrélés. Les logs ne gardent pas une transaction SQL ouverte pendant la durée du build.

Références de l'analyse

[monorepo · services/app-management/src/app_manager/domain/service/image/image_build_service.py:97](#) Source Git liée et résolue.

[monorepo · services/app-management/src/app_manager/domain/service/image/image_build_service.py:141](#) Persistance et publication du build.

[monorepo · services/app-management/src/app_manager/domain/service/image/image_build_service.py:322](#) Logs streamés ou stockés, transaction relâchée.

[monorepo · services/app-management/src/app_manager/application/router/image_router.py:194](#) Lecture du résultat de scan.

AM-05 **Cible non qualifiée****Dépôts externes, dépôts internes et édition de sources****COMPORTEMENT OBSERVÉ**

Les API distinguent dépôts externes liés aux applications et dépôts internes. Elles vérifient l'accès distant, listent les sources et permettent de lier/délier un dépôt. Le dépôt interne ajoute création, changement de projet, commits, branches, tags, arborescence, lecture/écriture/suppression et upload de fichier. Le service de fichiers résout le dépôt, associe l'auteur et délègue les opérations au client Git ; un renommage avec mise à jour passe par une opération groupée de fichiers. La source d'un build doit être un dépôt lié à l'application.

UTILISATEURS ET RÔLES

Développeur dans la Factory et administrateur Git.

INTERFACES DANS LE CODE

Routers /repositories, /repositories/internal et /apps/{app_id}/repositories ; InternalRepositoryFileService.

DONNÉES ET ÉTATS

RepositoryEntity, liens application-dépôt, visibilité, projet, branches/tags/commits Git et auteurs.

DÉPENDANCES ET ÉCHANGES

Gitea interne et autres fournisseurs Git, SSH, utilisateurs et IAM.

COMPORTEMENT CIBLE PROPOSÉ

Applications Rust porte la gestion des dépôts et effets Git, Flows épinglent les commits qu'il exécute. Gitea Go peut rester encapsulé. L'édition de source ne publie aucune édition de données.

MIGRATION DE CETTE CAPACITÉ

Préserver IDs, URLs non secrètes, commits et auteurs. Séparer l'accès de lecture à la source de la permission de build/déploiement. Conserver les préconditions Git de mise à jour.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une édition basée sur un fichier devenu obsolète retourne un conflit compréhensible. Changer le dépôt lié ne modifie pas le commit d'un build déjà admis.

Références de l'analyse

[monorepo · services/app-management/src/repository/application/router/internal_repository_router.py:209](#) Branches, tags et historique.

[monorepo · services/app-management/src/repository/application/router/repository_application_router.py:59](#) Rattachement à l'application.

[monorepo](#) · [services/app-management/src/repository/domain/service/internal_repository_file_service.py:154](#) Édition groupée et attribution.

[monorepo](#) · [services/app-management/src/repository/application/router/internal_repository_file_router.py:194](#) Contrats fichiers et arborescence.

AM-06 Cible non qualifiée

Devspace personnel, accès SSH et provisioning

COMPORTEMENT OBSERVÉ

La création d'un devspace vérifie l'utilisateur et l'absence d'espace déjà associé. Elle autorise les liens vers applications agents, réutilise registre/image, persiste application, instance et clé SSH puis assure l'accès au dépôt interne avant de lancer le déploiement. Les opérations sont donc réparties entre transaction SQL et effets distants. Les services distincts couvrent lecture, liste, mise à jour, suppression, authorized keys et logs de déploiement. Les clés privées sont chiffrées au repos selon le service crypto existant.

UTILISATEURS ET RÔLES

Développeur et administrateur des espaces.

INTERFACES DANS LE CODE

DevspaceRouter ; CreateDevspaceService.create_devspace ; UpdateDevspaceAuthorizedKeysService ; DevspaceSSHKeyService.

DONNÉES ET ÉTATS

Application de type DEVSPACE, instance, utilisateur, clé et déploiement.

DÉPENDANCES ET ÉCHANGES

Git interne, Kubernetes, Helm, image devspace, IAM et agent de tunnel.

COMPORTEMENT CIBLE PROPOSÉ

Applications/Runtimes Rust orchestre un espace isolé, quotas et reçus de provisioning. Les accès SSH/tunnel ont identité, durée et révocation propres ; le SDK données utilise les frontières Query/Writer.

MIGRATION DE CETTE CAPACITÉ

Importer clés par une procédure sécurisée sans export clair, préserver volumes et dépôts, puis recréer les espaces depuis des images sans JVM. Rendre chaque effet réconciliable.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Interrompre après persistance mais avant accès Git : la reprise complète le même espace sans doublon. Révoquer le développeur coupe les nouvelles sessions et retire les clés prévues.

Références de l'analyse

[monorepo](#) · [services/app-management/src/devspace/domain/service/create_devspace_service.py:96](#) Parcours SQL, clés, Git puis déploiement.

[monorepo](#) · [services/app-management/src/devspace/domain/service/create_devspace_service.py:207](#) Unicité et utilisateur.

[monorepo](#) · [services/app-management/src/devspace/application/router/devspace_router.py:51](#) Surface devspaces.

AM-07 Cible non qualifiée

Sessions de calcul Python et Spark pour process-data

COMPOTEMENT OBSERVÉ

Les routes privées start/delete/reconcile et jobs délèguent au SessionRuntimeServiceImpl. L'adaptateur rend des templates de pod pour Python, client Spark et serveur Spark Connect, applique services/deployments et réconcilie les objets observés. Le submit adresse le harness de la session ; le chemin concerné résout les credentials du projet avant l'appel. L'état et l'annulation utilisent le statement de cette session. Ces fonctions appartiennent au runtime de données bien qu'elles soient hébergées dans app-management.

UTILISATEURS ET RÔLES

Process-data et exploitant des runtimes.

INTERFACES DANS LE CODE

/private/sessions ; SessionRuntimeServiceImpl.start_sessions, start_connect_server, reconcile_clusters, submit_job, get_job_state, cancel_job.

DONNÉES ET ÉTATS

Ressources Kubernetes de sessions et serveurs ; statement du harness ; process-data conserve le run métier.

DÉPENDANCES ET ÉCHANGES

Kubernetes, templates runtime, credentials projet, Python/PySpark et Spark Connect.

COMPOTEMENT CIBLE PROPOSÉ

Runtimes Rust reprend cette frontière avec des workers fongibles et des tentatives bornées. Writer médie le staging ; Publisher contrôle la publication. Aucun serveur Spark ou composant JVM final.

MIGRATION DE CETTE CAPACITÉ

Conserver un adaptateur temporaire process-data puis déplacer admission, capacités et réconciliation dans Runtimes. Qualifier les scripts Python/SQL ; inventorier les incompatibilités PySpark.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Perdre la réponse du submit puis réconcilier ne produit pas deux publications. Une ancienne tentative ne peut écrire avec la capacité d'une nouvelle ; aucune permission REST générique table.modify ne parvient au worker.

Références de l'analyse

[monorepo · services/app-management/src/app_manager/application/router/private_session_router.py:37](#) Routes privées de sessions.

[monorepo · services/app-management/src/app_manager/infra/adapters/session/session_runtime_service_impl.py:213](#) Création des ressources runtime.

[monorepo · services/app-management/src/app_manager/infra/adapters/session/session_runtime_service_impl.py:453](#) Harness et credentials de projet.

[monorepo · services/app-management/src/app_manager/infra/adapters/session/session_runtime_service_impl.py:372](#) Inventaire et nettoyage.

AM-08 **Cible non qualifiée**

Catalogue d'applications agents et liens de communication

COMPOTEMENT OBSERVÉ

Un router dédié liste les applications agents et gère l'octroi/retrait d'accès entre applications. La création d'application valide chaque cible par la décision IAM de lecture car un lien peut ouvrir un accès réseau lors du déploiement. Les cibles inconnues, DEVSPACE ou refusées ne deviennent pas des liens valides. Cette relation

doit être traitée comme un changement de droit et de réseau, pas seulement comme une association d'affichage.

UTILISATEURS ET RÔLES

Auteur d'application, utilisateur métier et IAM.

INTERFACES DANS LE CODE

ApplicationAgentRouter.list_agents, grant_apps_access, revoke_app_access ; authorize_agent_targets.

DONNÉES ET ÉTATS

AppToAppLink, applications cibles et faits d'autorisation.

DÉPENDANCES ET ÉCHANGES

IAM, catalogue applicatif et NetworkPolicy rendu au déploiement.

COMPORTEMENT CIBLE PROPOSÉ

Applications/IAM/Egress Rust gèrent les consommateurs déclarés et relations d'appel. EXPOSURE décrit une consommation autorisée ; elle n'est pas un nouveau kind de publication.

MIGRATION DE CETTE CAPACITÉ

Importer les liens existants avec leur justification et vérifier droits sur chaque cible. Rendre la mise à jour réseau observable et révocable après retrait.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Le retrait d'un lien supprime le droit et converge vers la fermeture réseau. Un appelant ne peut déduire l'existence d'une application privée en essayant de la lier.

Références de l'analyse

[monorepo · services/app-management/src/app_manager/application/router/application_agent_router.py:55](#) Catalogue et opérations de liens.

[monorepo · services/app-management/src/app_manager/application/router/application_agent_router.py:183](#) Révocation explicite.

[monorepo · services/app-management/src/app_manager/domain/service/application/create_application_service.py:7](#) Validation des cibles avant création.

AM-09 Cible non qualifiée

Secrets, clés de build et limite de confiance des identités

COMPORTEMENT OBSERVÉ

Le service crypto chiffre les passwords basic-auth, credentials de registres, variables secrètes et clés SSH privées. Les builds internes et externes ont des secrets SSH distincts ; seule l'URL exacte du dépôt interne peut utiliser la clé du compte Gitea interne. Les clones écrivent leur URL dans les logs, ce qui interdit d'y embarquer des credentials. Le header X-Userinfo n'est fiable que derrière le proxy configuré qui l'écrase ; sinon l'identité vient du bearer. Ces contraintes sont documentées par le composant et doivent être préservées lors du remplacement du langage.

UTILISATEURS ET RÔLES

Application, builder, opérateur de secrets et gateway.

INTERFACES DANS LE CODE

CryptoService ; build_repository_init_container ; contrat TRUST_PROXY_HEADERS ; user keys router.

DONNÉES ET ÉTATS

Secrets chiffrés en SQL et secrets montés dans les jobs ; identité entrante séparée du propriétaire de ressource.

DÉPENDANCES ET ÉCHANGES

Gestion des clés, Kubernetes Secrets, Gitea, IAM/APISIX et registres.

COMPORTEMENT CIBLE PROPOSÉ

Applications/IAM/Egress Rust utilisent des références de secrets, rotation et délégations bornées. Les secrets de build interne restent inaccessibles aux dépôts externes ; pas d'algorithme crypto maison.

MIGRATION DE CETTE CAPACITÉ

Migrer les ciphertexts avec une procédure de rekey contrôlée et auditée. Tester les mêmes routes derrière et sans proxy ; ne jamais importer de secrets dans le dossier.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une URL externe ressemblant à Gitea n'obtient pas sa clé. Un faux X-Userinfo sur un pod qui ne fait pas confiance au proxy ne donne aucun droit. Les logs ne contiennent pas de credential.

Références de l'analyse

[monorepo](#) · [services/app-management/AGENTS.md:72](#) Contrat de chiffrement documenté.

[monorepo](#) · [services/app-management/AGENTS.md:79](#) Séparation des clés et risque du clone URL.

[monorepo](#) · [services/app-management/AGENTS.md:86](#) Limite de confiance du header.

AM-10 **Cible non qualifiée**

Exploitation, événements et migrations avant déploiement**COMPORTEMENT OBSERVÉ**

Les consumers de build et déploiement attendent la fin des watchers, ce qui exige un poll interval adapté. Le composant documente que NACK_ON_ERROR est nécessaire pour conserver un message échoué, car le simple auto_commit=False ne donne pas cette garantie FastStream. Les migrations Alembic sont exécutées par un hook avant install/upgrade en mode migrate, pendant que les anciens pods peuvent encore servir. Les routes health distinguent live/ready et regardent notamment la base et IAM. La politique automatique de migrations du monorepo ne couvre pas cet arbre src/migrations.

UTILISATEURS ET RÔLES

Exploitant Kubernetes et équipe de livraison.

INTERFACES DANS LE CODE

Build/deployment consumers ; /health/live ; /health/ready ; hook migration-job ; Alembic.

DONNÉES ET ÉTATS

Offsets, exécutions longues, migrations et statut de disponibilité.

DÉPENDANCES ET ÉCHANGES

PostgreSQL, Kafka, Helm, IAM et Kubernetes.

COMPORTEMENT CIBLE PROPOSÉ

Migration/Applications Rust conservent Expand/Migrate/Contract, transactions d'événement et opérateurs repris après incident. La livraison reste par digest attesté ; tous les hooks/builders sont inclus dans le contrôle zéro JVM.

MIGRATION DE CETTE CAPACITÉ

Ajouter les migrations Rust à l'enforcement, transférer les messages non traités sans double propriétaire et qualifier l'arrêt/reprise des watchers. Conserver les checks de readiness utiles.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un échec de watcher ne perd pas l'événement. Le binaire précédent reste compatible pendant Expand ; un échec de migration bloque l'activation sans casser les pods servant encore.

Références de l'analyse

[monorepo](#) · [services/app-management/AGENTS.md:58](#) Hook et coexistence ancien schéma/binaire.

[monorepo](#) · [services/app-management/AGENTS.md:66](#) Acquittement et watchers.

[monorepo](#) · [services/app-management/src/app_manager/application/router/healthcheck_router.py:161](#) Disponibilité.

Architecture et exploitation du service cible

Plan de contrôle Rust proposé

- Core Rust pour applications, dépôts, instances, devspaces et opérations suivies.
- Identité IAM, quotas Runtimes, révisions immuables, idempotence et reçus des effets.
- Séparer gestion applicative et sessions de calcul, actuellement réunies dans ce service.

Plan de données proposé

- Exécutants de build/déploiement encapsulés, aucun runtime JVM requis à l'état final.
- Espaces Python utilisateur isolés, volumes et tunnel ; workers de données sans autorité de publication.

Découpage du core Rust

- applications-api et registry de configuration.
- deployment-controller avec commande persistée, watcher et réconciliation.
- git-adapter, environment-build-controller et secret references.
- devspace-controller et tunnel-coordinator séparés du contenu utilisateur.

Identité, autorisation et isolation

- Préserver parenté des ressources, décisions sur les applications liées et usage des comptes de service.
- Séparer clés SSH internes/externes et conserver chiffrement/rotation des secrets.
- Un header proxy ne fait foi que dans la frontière de confiance explicitement configurée.

Exploitation et reprise

- Pré-install/pre-upgrade migrations Expand compatibles ; policy src/migrations à rendre effective.

- Réconcilier les opérations commit SQL puis événement et les watchers interrompus.
- Sauvegarder base, clés, dépôts et volumes avec procédure de restauration des effets déjà réalisés.

Séquence de migration

- Importer identités, parentés et historique sans changer les IDs.
- Ajouter commandes/reçus et comparer en observation passive.
- Qualifier builds et déploiements sans JVM, puis basculer par application.
- Déplacer sessions de calcul vers Runtimes et retirer les adaptateurs Python métier remplacés.

Écarts et décisions encore ouverts

- Idempotence sous réponse perdue de chaque effet Helm, Git et build à qualifier.
- Configurations clients, état réel des registres/volumes et performances non inspectés.
- Les citations AGENTS sur secrets et transport sont des contrats documentés ; cette mission n'a pas réaudité chaque branche du watcher.

Références de l'analyse

[monorepo](#) · [services/app-management/src/app_manager/domain/service/application/create_application_service.py:36](#) Cœur applicatif lu.

[monorepo](#) · [services/app-management/src/app_manager/infra/adapter/session/session_runtime_service_impl.py:132](#) Frontière runtime également présente.